



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

COMPUTAÇÃO DE ALTO DESEMPENHO

DOCENTE: CARLA DOS SANTOS SANTANA

E SAMUEL XAVIER DE SOUZA

DISCENTE: CLARA VIRGÍNIA NASCIMENTO SANTOS (20240002212)

Tarefa 17

1. ENUNCIADO

Implemente um programa MPI que calcule o produto $y = A \cdot x$, onde A é uma matriz $M \times N$ e x é um vetor de tamanho N . Divida a matriz A por linhas entre os processos com `MPI_Scatter`, e distribua o vetor x inteiro com `MPI_Bcast`. Cada processo deve calcular os elementos de y correspondentes às suas linhas e enviá-los de volta ao processo 0 com `MPI_Gather`.

Compare os tempos com diferentes tamanhos de matriz e número de processos. Fazer avaliação de eficiência e speedup a medida que aumenta o tamanho da matriz e quantidade de processos.

2. DISCUSSÃO

Nesta tarefa, foi implementado um programa em MPI para calcular o produto entre uma matriz e um vetor. Esse tipo de operação é muito comum em computação científica, simulações, métodos numéricos e também em aplicações de alto desempenho, pois envolve uma grande quantidade de operações repetitivas.

O cálculo solicitado é dado por:

$$y = A \cdot x$$

Isso significa que cada elemento do vetor y é calculado a partir da multiplicação de uma linha da matriz A pelo vetor x . Dessa maneira, para cada linha da matriz, é feita uma soma acumulada:

$$y[i] = A[i][0] * x[0] + A[i][1] * x[1] + \dots + A[i][N-1] * x[N-1]$$

Como cada linha da matriz pode ser calculada de forma independente, esse problema é adequado para paralelização. Assim, cada processo pode receber um conjunto de linhas da matriz A, calcular a parte correspondente do vetor y, e depois devolver esse resultado para o processo principal.

Diferente do OpenMP, que trabalha com múltiplas threads compartilhando a mesma memória, o MPI trabalha com múltiplos processos, e cada processo possui seu próprio espaço de memória. Por isso, os dados precisam ser enviados de um processo para outro através de comunicação explícita.

Para resolver essa tarefa, foram usadas três operações principais do MPI:

- `MPI_Scatter`: divide a matriz A entre os processos. Como a divisão é feita por linhas, cada processo recebe um bloco de linhas da matriz.
- `MPI_Bcast`: envia o vetor x inteiro para todos os processos, pois todos precisam dele para calcular sua parte do produto.
- `MPI_Gather`: junta os pedaços calculados do vetor y e envia tudo de volta para o processo 0.

Dessa maneira, o processo 0 fica responsável por criar a matriz A e o vetor x, distribuir os dados para os demais processos, receber os resultados parciais e montar o vetor y final.

Contudo, como o enunciado solicitou o uso de `MPI_Scatter` e `MPI_Gather`, foi necessário considerar uma restrição: o número de linhas M precisa ser divisível pela quantidade de processos. Caso isso não aconteça, a divisão das linhas não será exata. Para resolver casos não divisíveis, seria necessário usar `MPI_Scatterv` e `MPI_Gatherv`, mas não foi usado porque a questão pediu especificamente `MPI_Scatter` e `MPI_Gather`.

2.1. MPI

MPI significa Message Passing Interface, isto é, uma interface de passagem de mensagens. Ele é usado quando os processos não compartilham a mesma memória diretamente, então precisam se comunicar enviando e recebendo dados.

Nesta tarefa, o MPI é usado da seguinte maneira:

- 1º) O processo 0 cria a matriz A e o vetor x;
- 2º) A matriz A é dividida em blocos de linhas;
- 3º) Cada processo recebe uma parte da matriz com MPI_Scatter;
- 4º) Todos os processos recebem o vetor x inteiro com MPI_Bcast;
- 5º) Cada processo calcula sua parte do vetor y;
- 6º) O processo 0 recebe todas as partes calculadas com MPI_Gather;
- 7º) O tempo total é medido e usado para calcular speedup e eficiência.

2.2. Speedup E Eficiência

O Speedup é uma métrica usada para verificar se o aumento da quantidade de processos realmente melhorou o desempenho do programa.

$$\text{Speedup} = \text{Tempo com 1 processo} / \text{Tempo com N processos}$$

Assim:

- Speedup > 1: o paralelismo ajudou;
- Speedup = 1: o paralelismo não trouxe ganho;
- Speedup < 1: o paralelismo piorou o desempenho;
- Speedup = N: seria o speedup ideal, mas raramente acontece na prática.

A Eficiência mostra o quanto os processos foram bem aproveitados.

$$\text{Eficiência} = \text{Speedup} / \text{Número de processos}$$

Para apresentar em porcentagem:

$$\text{Eficiência (\%)} = (\text{Speedup} / \text{Número de processos}) * 100$$

Assim, mesmo que o speedup aumente, a eficiência pode diminuir. Isso acontece porque, ao aumentar a quantidade de processos, também aumenta

o custo de comunicação entre eles. No caso desta tarefa, existe comunicação no envio das linhas da matriz, no envio do vetor x e na coleta do vetor y . Portanto, o desempenho não depende apenas da quantidade de processos, mas também do tamanho da matriz e do custo de comunicação.

2.3. Etapas Para Resolução Da Tarefa

Etapas para a resolução da tarefa:

- 1º) Inicializar o MPI;
- 2º) Obter o rank de cada processo e a quantidade total de processos;
- 3º) Definir os valores de M e N ;
- 4º) Verificar se M é divisível pela quantidade de processos;
- 5º) Criar a matriz A e o vetor x no processo 0;
- 6º) Dividir as linhas da matriz entre os processos usando MPI_Scatter;
- 7º) Enviar o vetor x inteiro para todos os processos usando MPI_Bcast;
- 8º) Cada processo calcula sua parte do vetor y ;
- 9º) O processo 0 recebe os resultados usando MPI_Gather;
- 10º) Medir o tempo total da execução;
- 11º) Calcular speedup e eficiência;
- 12º) Testar com diferentes tamanhos de matriz e diferentes quantidades de processos;
- 13º) Analisar os resultados.

2.4. RESOLUÇÃO EM CÓDIGO DA TAREFA

Segue no apêndice.

2.5. OUTPUT

Segue no apêndice.

3. CONCLUSÃO

Com a implementação da tarefa, foi possível observar como o MPI pode ser usado para paralelizar o produto entre uma matriz e um vetor. A divisão da matriz por linhas é uma estratégia simples e eficiente, pois cada linha pode ser calculada de forma independente.

O uso do MPI_Scatter permitiu distribuir partes da matriz entre os processos, enquanto o MPI_Bcast foi necessário para garantir que todos os processos tivessem acesso ao vetor x completo. Em seguida, cada processo calculou sua parte do vetor y e o processo 0 reuniu os resultados com MPI_Gather.

A partir dos testes, espera-se observar que o speedup melhora quando a matriz aumenta, pois existe mais trabalho computacional para dividir entre os processos. Contudo, para matrizes pequenas, o custo de comunicação pode reduzir os ganhos, fazendo com que o paralelismo não seja tão eficiente.

Além disso, a eficiência tende a diminuir conforme a quantidade de processos aumenta, pois sempre existe um custo de comunicação e sincronização. Assim, mesmo que o tempo diminua, nem todos os processos são aproveitados de forma perfeita.

Portanto, é possível concluir que o MPI é uma ferramenta importante para computação de alto desempenho, principalmente em problemas grandes e divisíveis. Entretanto, o ganho real depende do tamanho do problema, da quantidade de processos e do custo de comunicação entre eles.

4. ANÁLISE DE CÓDIGO

Para a matriz 1200x1200, observou-se que o melhor desempenho ocorreu com 2 processos, em que o tempo caiu de 0.005537 segundos para 0.003375 segundos. Dessa forma, o speedup foi de 1.64x e a eficiência foi de 82.02%, indicando que, nesse caso, o uso de MPI conseguiu trazer ganho real de desempenho.

Contudo, ao aumentar a quantidade de processos, o desempenho não continuou melhorando proporcionalmente. Com 4 processos, o tempo aumentou para 0.009282 segundos, gerando speedup menor que 1. Isso significa que, nesse caso, o paralelismo piorou o desempenho. Com 6 processos, houve uma pequena melhora em relação à execução sequencial, mas ainda com baixa eficiência. Já com 8 processos, o speedup também ficou menor que 1.

Isso acontece porque a matriz 1200x1200 ainda é relativamente pequena para o custo de comunicação do MPI. Assim, o tempo gasto com MPI_Scatter, MPI_Bcast e MPI_Gather passa a ter bastante peso no tempo total. Portanto, para esse tamanho de matriz, usar muitos processos não compensa, pois o overhead de comunicação pode ser maior que o ganho obtido ao dividir o cálculo.

5. APÊNDICE

5.1. RESOLUÇÃO

```

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int rank;
    int quantidade_processos;

    // Inicializa o MPI
    MPI_Init(&argc, &argv);

    // rank é o número de cada processo
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // quantidade total de processos que estão rodando
    MPI_Comm_size(MPI_COMM_WORLD, &quantidade_processos);

    // Valores padrão, caso a pessoa rode sem passar M e N no terminal
    int M = 1200; // quantidade de linhas da matriz
    int N = 1200; // quantidade de colunas da matriz e tamanho do vetor x

    // Tempo de referência é o tempo com 1 processo
    // passa ele pelo terminal depois para calcular speedup e eficiência
    double tempo_referencia = 0.0;

    // Se a pessoa passar M e N no terminal, usa esses valores
    if (argc >= 3) {
        M = atoi(argv[1]);
        N = atoi(argv[2]);
    }

    // Se a pessoa passar o tempo de referência, usa ele também
    if (argc >= 4) {
        tempo_referencia = atof(argv[3]);
    }

    // Conferindo se M e N são válidos
    if (M <= 0 || N <= 0) {
        if (rank == 0) {
            printf("Erro: M e N precisam ser maiores que zero.\n");
        }

        MPI_Finalize();
        return 1;
    }

    // Como estamos usando MPI_Scatter, M precisa dividir certinho pela
    quantidade de processos

```

```

// EM=1200 e processos=4  cada processo pega 300 linhas
if (M % quantidade_processos != 0) {
    if (rank == 0) {
        printf("Erro: M precisa ser divisivel pela quantidade de
processos.\n");
        printf("M = %d, processos = %d\n", M, quantidade_processos);
        printf("Tente usar M = 1200, 2400 ou 4800.\n");
    }

    MPI_Finalize();
    return 1;
}

int linhas_por_processo = M / quantidade_processos;
int quantidade_elementos_local = linhas_por_processo * N;

// Ponteiros principais
double *matriz_A = NULL; // matriz completa, só existe no processo 0
double *vetor_x = NULL; // vetor x, vai existir em todos os processos
double *vetor_y = NULL; // vetor y completo, só existe no processo 0
double *matriz_local = NULL; // pedaço da matriz que cada processo
recebe
double *vetor_y_local = NULL; // pedaço do resultado calculado por cada
processo

// Todos os processos precisam do vetor x
vetor_x = (double *) malloc(N * sizeof(double));

// Cada processo recebe só algumas linhas da matriz
matriz_local = (double *) malloc(quantidade_elementos_local *
sizeof(double));

// Cada processo calcula só algumas posições do vetor y
vetor_y_local = (double *) malloc(linhas_por_processo *
sizeof(double));

// Só o processo 0 guarda a matriz completa e o vetor y completo
if (rank == 0) {
    matriz_A = (double *) malloc(M * N * sizeof(double));
    vetor_y = (double *) malloc(M * sizeof(double));
}

// Verificação simples de memória
int erro_memoria = 0;

if (vetor_x == NULL || matriz_local == NULL || vetor_y_local == NULL) {
    erro_memoria = 1;
}

```

```

    if (rank == 0 && (matriz_A == NULL || vetor_y == NULL)) {
        erro_memoria = 1;
    }

    // Se qualquer processo teve erro de memória, todos ficam sabendo
    int erro_global = 0;
    MPI_Allreduce(&erro_memoria, &erro_global, 1, MPI_INT, MPI_MAX,
MPI_COMM_WORLD);

    if (erro_global == 1) {
        if (rank == 0) {
            printf("Erro: faltou memoria para alocar os
vetores/matriz.\n");
        }

        free(vetor_x);
        free(matriz_local);
        free(vetor_y_local);

        if (rank == 0) {
            free(matriz_A);
            free(vetor_y);
        }

        MPI_Finalize();
        return 1;
    }

    // O processo 0 inicializa a matriz A e o vetor x
    if (rank == 0) {
        // Preenchendo a matriz com valores simples
        // Usei uma continha simples só pra não deixar tudo igual
        for (int i = 0; i < M; i++) {
            for (int j = 0; j < N; j++) {
                matriz_A[i * N + j] = (double) ((i + j) % 10 + 1);
            }
        }

        // Preenchendo o vetor x
        // tudo 1.0 pra facilitar a conferência do resultado
        for (int j = 0; j < N; j++) {
            vetor_x[j] = 1.0;
        }
    }

    // Sincroniza todo mundo antes de começar a medir o tempo
    MPI_Barrier(MPI_COMM_WORLD);

```



```

double inicio_total = MPI_Wtime();

// Divide a matriz A por linhas entre os processos
MPI_Scatter(matriz_A, quantidade_elementos_local,
MPI_DOUBLE, matriz_local, quantidade_elementos_local, MPI_DOUBLE,
0, MPI_COMM_WORLD);

// Envia o vetor x inteiro para todos os processos
MPI_Bcast(vetor_x, N, MPI_DOUBLE, 0, MPI_COMM_WORLD);

// Medindo só o tempo de cálculo local também
double inicio_calculo = MPI_Wtime();

// Cada processo calcula as linhas que recebeu
for (int i = 0; i < linhas_por_processo; i++) {
    double soma = 0.0;

    for (int j = 0; j < N; j++) {
        soma += matriz_local[i * N + j] * vetor_x[j];
    }

    vetor_y_local[i] = soma;
}

double fim_calculo = MPI_Wtime();

// Junta todos os pedaços do vetor y no processo 0
MPI_Gather(vetor_y_local,
linhas_por_processo, MPI_DOUBLE, vetor_y, linhas_por_processo, MPI_DOUBLE,
0, MPI_COMM_WORLD);

double fim_total = MPI_Wtime();

double tempo_total_local = fim_total - inicio_total;
double tempo_calculo_local = fim_calculo - inicio_calculo;

// Como cada processo pode terminar em tempos um pouco diferentes,
// pegamos o maior tempo, pois o programa só termina quando todos
terminam
double tempo_total = 0.0;
double tempo_calculo = 0.0;

MPI_Reduce(&tempo_total_local, &tempo_total, 1, MPI_DOUBLE, MPI_MAX, 0,
MPI_COMM_WORLD);
MPI_Reduce(&tempo_calculo_local, &tempo_calculo, 1, MPI_DOUBLE,
MPI_MAX, 0, MPI_COMM_WORLD);

```

```

// Processo 0 imprime os resultados finais
if (rank == 0) {
    double checksum = 0.0;

    // Soma o vetor y só pra garantir que o resultado foi realmente
    usado
    // e também pra comparar se dá o mesmo valor com vários processos
    for (int i = 0; i < M; i++) {
        checksum += vetor_y[i];
    }

    printf("\nRESULTADO\n");
    printf("Matriz A: %d x %d\n", M, N);
    printf("Processos: %d\n", quantidade_processos);
    printf("Linhas por processo: %d\n", linhas_por_processo);
    printf("Tempo total: %f segundos\n", tempo_total);
    printf("Tempo apenas do calculo local: %f segundos\n",
tempo_calculo);
    printf("Checksum do vetor y: %.2f\n", checksum);

    // Se a pessoa passou o tempo de referência, calcula speedup e
    eficiencia
    if (tempo_referencia > 0.0) {
        double speedup = tempo_referencia / tempo_total;
        double eficiencia = (speedup / quantidade_processos) * 100.0;

        printf("Tempo de referencia com 1 processo: %f segundos\n",
tempo_referencia);
        printf("Speedup: %.2fx\n", speedup);
        printf("Eficiencia: %.2f%%\n", eficiencia);
    } else {
        printf("Tempo de referencia nao informado.\n");
        printf("Se este teste foi com 1 processo, use o Tempo total
como referencia nos proximos testes\n");
    }
}

// free
free(vetor_x);
free(matriz_local);
free(vetor_y_local);

if (rank == 0) {
    free(matriz_A);
    free(vetor_y);
}

// Finaliza o MPI

```

```

    MPI_Finalize();

    return 0;
}

```

5.2. OUTPUT

```

PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa17> mpiexec -n 2 .\tarefa17.exe 1200 1200 0.005537
>> mpiexec -n 4 .\tarefa17.exe 1200 1200 0.005537
>> mpiexec -n 6 .\tarefa17.exe 1200 1200 0.005537
>> mpiexec -n 8 .\tarefa17.exe 1200 1200 0.005537

```

```

RESULTADO
Matriz A: 1200 x 1200
Processos: 2
Linhas por processo: 600
Tempo total: 0.003375 segundos
Tempo apenas do calculo local: 0.000464 segundos
Checksum do vetor y: 7920000.00
Tempo de referencia com 1 processo: 0.005537 segundos
Speedup: 1.64x
Eficiencia: 82.02%

```

```

RESULTADO
Matriz A: 1200 x 1200
Processos: 4
Linhas por processo: 300
Tempo total: 0.009282 segundos
Tempo apenas do calculo local: 0.000712 segundos
Checksum do vetor y: 7920000.00
Tempo de referencia com 1 processo: 0.005537 segundos
Speedup: 0.60x
Eficiencia: 14.91%

```

```

RESULTADO
Matriz A: 1200 x 1200
Processos: 6
Linhas por processo: 200
Tempo total: 0.004730 segundos
Tempo apenas do calculo local: 0.000441 segundos
Checksum do vetor y: 7920000.00
Tempo de referencia com 1 processo: 0.005537 segundos
Speedup: 1.17x
Eficiencia: 19.51%

```

```

RESULTADO

```

```

RESULTADO
Matriz A: 1200 x 1200
Processos: 8
Linhas por processo: 150
Tempo total: 0.006019 segundos
Tempo apenas do calculo local: 0.000349 segundos
Checksum do vetor y: 7920000.00
Tempo de referencia com 1 processo: 0.005537 segundos
Speedup: 0.92x
Eficiencia: 11.50%

```

```

PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa17>

```

